

The DesiredGainR Breeder's Guide

Turning breeder intent into a defensible selection decision — Edition 1

Félicien Akohoue · PhD

05 August 2026

Contents

Executive summary	3
1 Purpose and scope	4
1.1 What the package does	4
1.2 What it deliberately does not do	4
1.3 What <code>n_select</code> does - and does not do	4
1.4 Feature maturity	5
2 Before any recommendation	5
2.1 Minimum evidence	5
2.2 Selection information and objective traits	5
2.3 Covariance provenance and admissibility	6
2.4 Conditioning	6
2.5 Repairing an inadmissible matrix	6
2.6 Units and direction	7
3 Defining the objective	7
3.1 Why economic weights are difficult	7
3.2 The two directions of conversion	7
3.3 Is the objective attainable?	8
3.4 How much does the answer depend on the numbers?	8
3.5 What was the programme actually selecting for?	8
4 Understanding multiple-trait selection	9
4.1 Definition and purpose	9
4.2 The four broad strategies	9
4.3 The common index model	10
4.4 From a breeding statement to a method	10

5	Methods implemented in DesiredGainR	10
5.1	Weight-based families	10
5.1.1	selection_index(method = "smith_hazel")	10
5.1.2	selection_index(method = "base")	11
5.2	Desired-gain families	11
5.2.1	selection_index(method = "pesek_baker") and "yamada"	11
5.3	Rank and threshold families	11
5.3.1	selection_index(method = "mulamba_mock")	11
5.3.2	selection_index(method = "independent_culling")	11
5.3.3	selection_index(method = "elston")	12
5.3.4	selection_index(method = "tandem")	12
5.4	Constrained families	12
5.5	Iterative optimisation of the desired-gain index	13
5.5.1	run_dgsi()	13
5.6	The quadratic genomic index	14
5.6.1	run_qgsi()	14
6	Evaluating and comparing indices	14
6.1	What a comprehensive comparison requires	14
6.2	Match the evaluation to the method	15
6.3	The classical criteria	15
6.4	How each fails	17
6.5	A stronger decision panel for desired-gain indices	17
6.6	Reusing a fitted index	17
7	Uncertainty	17
7.1	Why this section is not optional	17
7.2	index_uncertainty()	18
7.2.1	Choosing the degrees of freedom	18
7.2.2	Reading the result	18
7.3	propagate_covariance_uncertainty()	18
8	Multi-environment structure	19
9	Simulation evidence	19

10 End-to-end pipeline walkthrough	20
10.1 Phase 1 — Evidence	20
10.2 Phase 2 — Objective	20
10.3 Phase 3 — Index and ranking	20
10.4 Phase 4 — Uncertainty and sign-off	21
11 Interpreting diagnostics	21
11.1 Conditioning and stability	21
11.2 Selection and shortfall	21
12 Decision sign-off	21
13 Complete function map	22
13.1 Guide access	22
13.2 Objective definition	22
13.3 Covariance handling	22
13.4 Index construction	22
13.5 Evaluation and reuse	23
13.6 Uncertainty	23
13.7 Multi-environment	23
13.8 Simulation	23
13.9 Reference values	24
References	24

Executive summary

DesiredGainR converts a breeder’s stated intent into index coefficients, a candidate ranking, and the diagnostics needed to defend the resulting decision. Its distinctive premise is that most programmes cannot supply economic weights but can state what they want to change, and that the two are convertible.

Four claims govern how the package should be read.

An objective is a hypothesis about attainability, not a target. Response is bounded by selection intensity and by available genetic variation. A stated set of desired gains is frequently impossible at any intensity the programme can run, and that is checkable before an index is built.

Every reported gain conditions on covariance matrices that are estimates. Multi-trait restricted maximum likelihood usually estimates more variance components than the trial has independent families to determine. Results that do not report how far they move under that uncertainty are incomplete.

Constrained objectives usually have closed-form answers. Where a restriction on the response is linear, searching for it is unnecessary and introduces a dependence on the number of replicates run.

A decision must remain defensible after the fact. The package records the provenance of every covariance matrix, refuses inadmissible inputs rather than reporting impossible quantities, and reports repairs rather than performing them silently.

No R knowledge is required to read this guide. The companion vignette *Defining a breeding objective* covers the same ground with runnable code.

1 Purpose and scope

1.1 What the package does

DesiredGainR sits between a fitted multi-trait model and an index decision. It takes covariance matrices and candidate values as given, and returns coefficients, a ranking, expected responses and diagnostics.

Its job ends there. The ranking is evidence for the next decision, not a final crossing plan. Use HapBlockR to choose the operational parent set, perform optimal contribution selection (OCS), rank candidate crosses and allocate matings. HapBlockR’s `build_selection_index()` calls DesiredGainR when DGSI or QGSI is requested, then carries the resulting merit score into its parent- and cross-selection tools. DesiredGainR never calls HapBlockR.

OCS and optimum cross selection answer different questions. OCS determines how much each parent should contribute while controlling coancestry. Optimum cross selection determines which parent pairs to make and how many matings to assign under cross merit, diversity and operational constraints. HapBlockR owns both; DesiredGainR supplies the multi-trait merit objective they can use.

1.2 What it deliberately does not do

Task	Why not	Use instead
Choose the final parent set	A top index rank does not account for relatedness, complementarity or representation	<code>HapBlockR::select_parents_ga_ts()</code> or another HapBlockR parent-selection method
Control contributions and inbreeding	OCS is a population-management problem, not an index-coefficient problem	<code>HapBlockR::select_parents_ocs()</code> with a true OCS engine
Perform optimum cross selection and allocate matings	Pair performance and crossing constraints are different from individual merit	<code>HapBlockR::usefulness_criterion()</code> and HapBlockR mating-plan tools
Fit the mixed model	Estimating G and P is upstream work	ASReml-R, sommer, breedR, StageWise. Import with <code>import_covariance()</code> .
Simulate a breeding programme	The simulation layer is narrow by design: it compares desired-gain directions, nothing more	AlphaSimR directly

1.3 What `n_select` does - and does not do

In DesiredGainR, `n_select` sets the truncation proportion. From that proportion the package obtains selection intensity, calculates predicted or realised response and identifies a top-ranked set for stability comparisons. It is an analytical input, not a parental-selection algorithm. Do not read “top 20” as “cross these 20”. HapBlockR may retain a different set after it accounts for genomic relationships, contributions, complementary haplotypes, cross usefulness and programme constraints.

1.4 Feature maturity

Not every layer is equally established. Treating them alike invites a decision to be defended on the weakest one.

Maturity	Components
Stable	Objective definition, feasibility, covariance diagnostics, bending and import
Beta	Classical index families, restricted and proportional-gain indices, the desired-gain index and the quadratic genomic index
Experimental	Covariance uncertainty propagation
Experimental, not for operational recommendation	Recurrent simulation, direction optimisation, diversity optimisation and clonal mode

2 Before any recommendation

2.1 Minimum evidence

A responsible index requires all of the following.

Requirement	Why it matters	Failure mode if absent
Candidate values on a stated scale	The index is a linear combination. Scale determines what it weights.	Coefficients that silently favour whichever trait carries the largest variance
A genetic covariance $\mathbf{G}$	Determines what selection transmits	No expected response can be computed
A phenotypic covariance $\mathbf{P}$	Determines what selection acts on	Accuracy and heritability undefined
Favourable direction per trait	Which way is better is not inferable from the data	An index that selects against the objective
Units per trait	Needed to interpret any gain	Gains with no clear link to programme targets

2.2 Selection information and objective traits

The information used to rank candidates can differ from the genetic objective. This distinction matters after a multi-trait or multi-environment mixed model. The information can include records from relatives, environments, testcrosses, or correlated traits. The objective can remain a smaller set of genetic quantities.

Let $\mathbf{x}$ contain the selection information. Let $\mathbf{g}$ contain the objective traits. Define

$$\mathbf{P} = \text{Var}(\mathbf{x}), \quad \mathbf{C} = \text{Cov}(\mathbf{x}, \mathbf{g}), \quad \mathbf{G} = \text{Var}(\mathbf{g}).$$

For economic weights $\mathbf{a}$,

$$\mathbf{b} = \mathbf{P}^{-1}\mathbf{C}\mathbf{a}.$$

For a desired-gain direction $\mathbf{d}$,

$$\mathbf{b} = \mathbf{P}^{-1}\mathbf{C}(\mathbf{C}^{\top}\mathbf{P}^{-1}\mathbf{C})^{-1}\mathbf{d}.$$

Use `selection_information()` to declare these matrices. It checks their joint covariance. Use `generalized_index()` to fit the index.

A desired gain stated in genetic standard deviations requires conversion to the units of $\mathbf{G}$. Multiply each entry by $\sqrt{\text{diag}(\mathbf{G})}$. Then declare lower-is-better traits in `generalized_index()`. This keeps units and signs explicit.

A genomic estimated breeding value (GEBV) is a prediction. Its prediction error variance (PEV) measures uncertainty for one trait. Prediction error covariance (PEC) extends this quantity across traits. Candidate-specific PEC matrices support score intervals and selection probabilities through `candidate_score_uncertainty()`.

2.3 Covariance provenance and admissibility

$\mathbf{G}$ is a component of $\mathbf{P}$, so the residual $\mathbf{P} - \mathbf{G}$ is itself a covariance matrix and must be positive semidefinite. This is not a formality. Every consequence of violating it is a number that looks valid and is impossible: a heritability above one, an accuracy above one, a squared correlation above one, a negative mean squared prediction error.

Checking the diagonal is **not sufficient**. The condition $\mathbf{G}_{jj} \leq \mathbf{P}_{jj}$ for every trait is necessary but not sufficient, because a linear combination of traits can have genetic variance exceeding its phenotypic variance while no single trait does. The eigenvalues of the difference are the correct test, and the package applies it.

When the pair fails, the message names the offending combination and the largest single-trait ratio, so that a correlation-driven failure is distinguishable from a variance-driven one.

2.4 Conditioning

Report the condition number of $\mathbf{G}$ before trusting any index built on original trait scales.

$$\kappa(\mathbf{G}) = \frac{\lambda_{\max}}{\lambda_{\min}}$$

A large value means the matrix is close to singular and the coefficients are unstable: small changes in the estimate produce large changes in the index. Standardise the traits and look again. `matrix_diagnostics()` reports κ , the eigenvalues, the numerical rank and positive definiteness.

2.5 Repairing an inadmissible matrix

Multi-trait restricted maximum likelihood (REML) routinely returns a $\mathbf{G}$ that is not positive definite. `bend_covariance()` repairs it through `ASRgenomics::G.tuneup()`, which is a wrapper around Higham's alternating-projections algorithm.

Two of that function's fixed choices matter and are surfaced in the result:

Setting	Consequence
<code>keepDiag = FALSE</code>	Trait variances are not preserved, so the heritabilities implied by the bent matrix are not those supplied. Read <code>max_variance_change</code> .
<code>posd.tol = 0.01</code>	Eigenvalues floored at a hundredth of the largest, capping κ near 100. This is appropriate for an $n \times n$ relationship matrix. It is aggressive for a $p \times p$ trait covariance, where κ of 10^3 to 10^4 is ordinary and genuine.

Bending is a repair of an estimate, not an estimate. Report the adjustment alongside any result derived from a bent matrix.

2.6 Units and direction

Every trait needs a declared favourable direction. Traits where smaller is better — disease severity, days to maturity, plant height where reduction is wanted — are named in `lower_is_better`, and the package reverses their sign internally so that larger always means better in the analysis space.

Desired gains are stated as **non-negative magnitudes** in that favourable-direction space. Economic weights can be positive, zero or negative. A negative economic weight can arise when correlated response would otherwise move a trait beyond its preferred level. Trait direction remains declared through `lower_is_better`.

3 Defining the objective

3.1 Why economic weights are difficult

The classical index needs a price for every trait: how much a unit of grain yield is worth relative to a unit of plant height. In principle these are economic quantities. In practice almost no programme has them, and two people who agree entirely about the intent will write down different weights.

What programmes have is an intent: *lift yield without letting the cycle get longer*. Another example is *hold disease resistance where it is and push everything else*. An intent is not a weight vector.

3.2 The two directions of conversion

The package treats the intent as the input and the weights as derived.

Forward. Given economic weights $\mathbf{a}$, the Smith–Hazel coefficients are

$$\mathbf{b} = \mathbf{P}^{-1}\mathbf{G}\mathbf{a}$$

and the expected correlated response is $\Delta = i \mathbf{G}\mathbf{b}/\sqrt{\mathbf{b}^T\mathbf{P}\mathbf{b}}$.

Backward. Given desired gains $\mathbf{d}$, the weight vector for which those gains *are* the Smith–Hazel optimum is

$$\mathbf{w} = \mathbf{G}^{-1}\mathbf{P}\mathbf{G}^{-1}\mathbf{d}$$

and conversely the gains implied by weights $\mathbf{w}$ are $\mathbf{d} = \mathbf{G}\mathbf{P}^{-1}\mathbf{G}\mathbf{w}$.

`implied_economic_weights()` and `implied_desired_gains()` perform these conversions. The implied weights reproduce a coefficient direction under the supplied covariance matrices. They do not estimate prices or profit. Read them for **sign surprises**: a negative implied weight on a trait you asked to improve means the index obtains more of that trait by selecting against it directly, because of how strongly it correlates with something else in the objective. That may be correct, but it should be a decision rather than an accident.

If the desired gains are stated in genetic standard deviations while $\mathbf{G}$ and $\mathbf{P}$ remain in original trait units, the returned weights are per original unit. Hence, compare `weight * genetic_sd`, not the raw coefficients. This multiplication converts the weights to a common per-genetic-standard-deviation scale. It does not standardise the desired gains again. If the covariance matrices have already been standardised, do not apply the multiplication a second time.

3.3 Is the objective attainable?

Desired gains have a direction and a magnitude. Neither should be assumed attainable before the covariance structure and planned selection intensity are examined.

Let $\mathbf{b}$ denote the index coefficients, $\mathbf{G}$ the additive genetic covariance matrix, $\mathbf{P}$ the phenotypic covariance matrix, and i the standardised selection intensity. The expected one-cycle response is

$$\mathbf{R} = i \frac{\mathbf{G}\mathbf{b}}{\sqrt{\mathbf{b}^T \mathbf{P} \mathbf{b}}}.$$

The set of responses attainable at selection intensity i is the ellipsoid

$$\mathbf{R}^T \mathbf{G}^{-1} \mathbf{P} \mathbf{G}^{-1} \mathbf{R} = i^2,$$

so the intensity required to attain a stated $\mathbf{d}$ is

$$i_{\text{required}} = \sqrt{\mathbf{d}^T \mathbf{G}^{-1} \mathbf{P} \mathbf{G}^{-1} \mathbf{d}}.$$

`gain_feasibility()` reports: (i) the required intensity, (ii) the continuous selected proportion associated with that intensity, (iii) whether the planned selection proportion is sufficient, (iv) whether even selecting one candidate from the finite population is sufficient, and (v) the response available at the planned intensity in the same units as the input.

For example, a required proportion of 0.1773% among 200 candidates represents 0.355 candidate. It is not rounded to one, because selecting one candidate retains 0.5% and gives a lower intensity. A reported attainable fraction of 54.7% means that 54.7% of **every component** is available when the exact trait ratio is preserved.

Keep this exact-ray test separate from a minimum-floor objective. If a breeder accepts gains larger than the stated minimums, another response vector may meet all floors without preserving their ratio. Therefore: (i) scale the whole vector when its ratio is binding, or (ii) use population-driven or interval optimisation when the entries are trait-specific minimums, and assess the worst-trait and joint-attainment results.

The conclusion is conditional on a linear index, one-cycle truncation selection, approximate normality, and the supplied covariance matrices. Check covariance uncertainty and selected-set stability before issuing a recommendation.

3.4 How much does the answer depend on the numbers?

`weight_sensitivity()` perturbs the objective and reports how far the selected set moves.

Outcome	Reading
Selected set stable under a 20% weight change	The recommendation is defensible in a meeting
Selected set moves substantially	The data cannot distinguish your objective from a neighbouring one. Decide on other grounds.

3.5 What was the programme actually selecting for?

`retrospective_weights()` recovers the weight vector implied by a decision already made, from the observed selection differential $\mathbf{s}$:

$$\mathbf{b} = \mathbf{P}^{-1} \mathbf{s}$$

Comparing the recovered objective against the stated one is often the only way to discover what a programme's objective has actually been.

4 Understanding multiple-trait selection

4.1 Definition and purpose

Multiple-trait selection is the choice of candidates using two or more traits. The traits may enter the decision together or at different breeding stages. The purpose is to improve overall cultivar value while respecting genetic, biological and commercial trade-offs.

The objectives of this chapter are to: (i) define the main strategies, (ii) explain the index quantities, (iii) connect each breeding question to a method, and (iv) establish a fair comparison procedure.

4.2 The four broad strategies

Beavis, Lamkey, Mahama and Suza (2023) distinguish four broad strategies.

Strategy	Meaning	Suitable situation	Main risk
Multistage selection	Different traits are evaluated at different stages.	Expensive traits follow inexpensive screening.	Early rejection can remove candidates with later value.
Tandem selection	One trait is improved across one or more cycles before another trait becomes the focus.	One trait has temporary programme priority.	Other traits can deteriorate before the focus changes.
Independent culling	Every candidate must satisfy every stated limit.	Release standards or biological limits are firm.	Small failures receive no compensation from other traits.
Index selection	Traits are combined into one measure of overall merit.	Simultaneous improvement and explicit trade-offs are required.	The decision depends on the stated objective and input quality.

These strategies solve different problems. Therefore, method choice starts with the allowed trade-offs. It does not start with the most elaborate algorithm.

Use four questions to choose the strategy.

1. Are any limits non-negotiable? Apply independent culling to those traits.
2. Are traits measured at different costs or stages? Use a multistage design.
3. Must one trait dominate several cycles? Use a classical tandem schedule.
4. Can traits compensate for one another? Use index selection for those trade-offs.

Strategies can be combined. A breeder can cull lines that fail a disease limit and then rank the survivors with a desired-gain index.

DesiredGainR implements independent culling and several index families. The option `selection_index(method = "tandem")` is a within-cohort sequential screen. It is not classical tandem selection across cycles. The package has no dedicated multistage allocation optimiser or alternating-cycle tandem simulator in version 0.5.0.

4.3 The common index model

Let $\mathbf{x}$ contain the information used for selection. Let $\mathbf{g}$ contain the additive genetic values of the objective traits. Define

$$\mathbf{P} = \text{Var}(\mathbf{x}), \quad \mathbf{C} = \text{Cov}(\mathbf{x}, \mathbf{g}), \quad \mathbf{G} = \text{Var}(\mathbf{g}).$$

$\mathbf{P}$ describes the information variables. $\mathbf{C}$ describes how they predict the objective traits. $\mathbf{G}$ describes the available genetic variation. The selection index is

$$I = \mathbf{b}^T \mathbf{x}.$$

Its expected response is

$$\Delta = i \frac{\mathbf{C}^T \mathbf{b}}{\sqrt{\mathbf{b}^T \mathbf{P} \mathbf{b}}},$$

where i is the selection intensity. When information and objective traits are identical, $\mathbf{C} = \mathbf{G}$. When they differ, use `selection_information()` and `generalized_index()`.

4.4 From a breeding statement to a method

Breeder's statement	Starting method
I can defend relative economic values.	Smith-Hazel
I can state a desired response direction.	Pesek-Baker or Yamada
I can state acceptable desired-gain intervals.	<code>suggest_desired_gains()</code>
Every selected candidate must satisfy these limits.	Independent culling
Eligible candidates should have balanced margins.	Elston
Some traits should remain unchanged.	Restricted index
I need a simple comparator with few assumptions.	Base or Mulamba-Mock
Merit is genuinely curved or interactive.	Quadratic genomic selection index

5 Methods implemented in DesiredGainR

Distinguish the biological question from the optimisation method. A more elaborate method is not automatically more appropriate. Use the simplest method that represents the programme's actual objective and constraints.

5.1 Weight-based families

5.1.1 `selection_index(method = "smith_hazel")`

Question: *given prices for every trait, what maximises net merit?*

$$\mathbf{b} = \mathbf{P}^{-1} \mathbf{G} \mathbf{a}$$

The optimum index when $\mathbf{a}$ is genuinely known. Requires $\mathbf{G}$, $\mathbf{P}$ and economic weights.

5.1.2 `selection_index(method = "base")`

Question: *what is the simplest defensible benchmark?*

$$\mathbf{b} = \mathbf{a}$$

The weights are applied directly to the trait values with no covariance adjustment. Requires no $\mathbf{P}$. Always worth computing: if the optimum index barely beats the base index, the covariance structure is not doing much work and the extra estimation risk may outweigh the small advantage.

5.2 Desired-gain families

5.2.1 `selection_index(method = "pesek_baker")` and `"yamada"`

Question: *given a direction I want to move in, what index delivers it?*

$$\mathbf{b} = \mathbf{G}^{-1}\mathbf{d} \quad \text{and} \quad \mathbf{b} = \mathbf{P}^{-1}\mathbf{G}(\mathbf{G}\mathbf{P}^{-1}\mathbf{G})^{-1}\mathbf{d}$$

These are the same index. For invertible $\mathbf{G}$,

$$\mathbf{P}^{-1}\mathbf{G}(\mathbf{G}\mathbf{P}^{-1}\mathbf{G})^{-1}\mathbf{d} = \mathbf{P}^{-1}\mathbf{G}\mathbf{G}^{-1}\mathbf{P}\mathbf{G}^{-1}\mathbf{d} = \mathbf{G}^{-1}\mathbf{d}.$$

They are not alternatives to be compared, and a difference between them indicates numerical instability rather than a substantive choice.

The defining property is that the expected response is **exactly proportional to $\mathbf{d}$** :

$$\Delta = i \frac{\mathbf{G}\mathbf{b}}{\sqrt{\mathbf{b}^T\mathbf{P}\mathbf{b}}} = \frac{i}{\sqrt{\mathbf{b}^T\mathbf{P}\mathbf{b}}} \mathbf{d}.$$

This is what makes the family useful and what makes its estimation error measurable. See the chapter on uncertainty below.

5.3 Rank and threshold families

5.3.1 `selection_index(method = "mulamba_mock")`

Question: *the traits are on incomparable scales and I distrust the covariances — can I still combine them?*

Candidates are ranked within each trait and the ranks summed, optionally weighted. Uses no covariance matrix at all, so it is immune to a poorly estimated $\mathbf{G}$ and cannot report an expected genetic response.

5.3.2 `selection_index(method = "independent_culling")`

Question: *some traits are pass or fail, not tradeable.*

A candidate failing **any** threshold is never selected. `n_select` is a maximum, not a quota: if seven candidates pass and twenty were requested, seven are returned and the shortfall is reported with the gates each failure violated.

Supply thresholds in the original trait units. Use a minimum when larger values are favourable. Use a maximum for every trait listed in `lower_is_better`. `DesiredGainR` applies direction, centring and scaling internally.

This is deliberate. Filling the shortfall would mean trading a failed trait against a good one, which is what an index does — so if that is what is wanted, use an index.

5.3.3 `selection_index(method = "elston")`

Question: *every candidate must pass the floors. How should the survivors be ranked?*

The Elston index first applies every trait floor. It then multiplies each eligible candidate's margins above those floors. DesiredGainR uses the sum of log margins for numerical stability. A large margin in one trait cannot hide a failed floor in another trait.

This method is a useful weight-light comparator. Its closed-form response theory is limited. Inspect the selected differential. Use simulation when the method changes the operational decision.

5.3.4 `selection_index(method = "tandem")`

Question: *how would sequential within-cohort screening behave?*

This option filters one candidate cohort through traits in a stated order. It is an operational sequential-screen comparator. Classical tandem selection changes the focal trait across breeding cycles or generations. That strategy requires a multi-cycle schedule and is not implemented by this one-call option.

5.4 Constrained families

`restricted_index()` implements closed-form constraints on model-based expected response. `run_dgsi()` instead searches against the differential observed in a finite candidate set. The two procedures answer different questions and can select different candidates.

Let $\mathbf{C}$ be the $k \times p$ matrix selecting the constrained traits. Expected response is proportional to $\mathbf{G}\mathbf{b}$, so a zero-response restriction is $\mathbf{C}\mathbf{G}\mathbf{b} = \mathbf{0}$, and

$$\mathbf{b} = [\mathbf{I} - \mathbf{P}^{-1}\mathbf{G}\mathbf{C}^T(\mathbf{C}\mathbf{G}\mathbf{P}^{-1}\mathbf{G}\mathbf{C}^T)^{-1}\mathbf{C}\mathbf{G}] \mathbf{P}^{-1}\mathbf{G}\mathbf{a}.$$

Method	Question	Constraint
<code>kempthorne_nordskog</code>	Hold these traits still	Zero expected response in the restricted traits
<code>restricted_smith_hazel</code>	As above, projection form	Algebraically identical. It is retained because most texts present this form.
<code>tallis</code>	Move these traits in this ratio	Responses held in the supplied proportions. Magnitude is free.
<code>mallard</code>	Move these traits by these amounts	Responses held at specified values. Magnitude is fixed.
<code>harville</code>	Weight the constraint against merit	Restriction applied as a penalty. penalty = Inf recovers Kempthorne–Nordskog.

A restriction can never raise the correlation with net merit. The question is whether the loss is acceptable, and that is a breeding decision rather than a statistical one. Compare R_{HI} between the restricted and unrestricted indices to price it.

Satoh (2024) gives the response a useful geometric interpretation. For a complete proportional direction $\mathbf{d}$, the restricted breeding value is

$$\mathbf{g}_R = \mathbf{d}(\mathbf{d}^\top \mathbf{G}^{-1} \mathbf{d})^{-1} \mathbf{d}^\top \mathbf{G}^{-1} \mathbf{g} = \beta \mathbf{d}.$$

The scalar β measures progress along the requested direction. `restricted_breeding_values()` calculates it for every candidate. `evaluate_restricted_response()` applies the same projection to an achieved response. It also reports the Mahalanobis alignment and residual distance. Satoh showed that the Yamada proportional desired-gain index is a special case of restricted index theory.

5.5 Iterative optimisation of the desired-gain index

5.5.1 `run_dgsi()`

The desired-gain index formula remains unchanged. `run_dgsi()` adds the iterative search applied by Joukhadar et al. (2024). It searches for an input desired-gain vector whose selected-candidate differential best matches the breeder's target. It then reports the resulting coefficients.

DGSI is **not** a method hidden inside `selection_index()`. The call `selection_index(method = "yamada")` fits the closed-form, non-iterated desired-gain index. `run_dgsi()` starts from that coefficient direction, searches alternative directions and stores the original comparator in `non_iterated`. The dedicated *Iterative optimisation of the desired-gain index* vignette demonstrates the requested direction, feasibility, both responses, candidate ranking, full-sample comparison, holdout choice and stability diagnostics.

Variant	Argument	Use
Selection rule	<code>select_mode = "top_n"</code>	Rank on the index alone
	<code>select_mode = "eligible_top_n"</code>	Apply trait floors first, then rank
Replicate choice	<code>replicate_selection = "holdout"</code>	Default. Chooses the winning search on candidates held out of that comparison, so the choice does not depend on the candidates finally selected
	<code>replicate_selection = "training"</code>	Earlier behaviour. This is a minimum over <code>n_rep</code> draws and is biased downward.
Estimated P	<code>allow_incompatible_estimated_P</code>	Continue when a P estimated from reference candidates falls below G by more than sampling error explains. Recorded as an override

`run_dgsi()` reports two different quantities. Interpret them separately:

Quantity	Meaning
<code>realised_response</code>	The standardised differential between the selected candidates and the whole set. A property of who was picked
<code>theoretical_response</code>	$i \mathbf{G} \mathbf{b} / \sqrt{\mathbf{b}^\top \mathbf{P} \mathbf{b}}$ — what the next generation inherits

A large differential with a small transmitted response means selection is acting mostly on non-heritable variation.

The `dg` argument and `realised_response` use candidate standard-deviation units. A target stated in genetic standard deviations needs an explicit conversion. Multiply each target by its genetic standard deviation. Then divide by the standard deviation of that trait among the candidates. The iterative desired-gain vignette shows this calculation. This conversion keeps one biological target consistent across the feasibility test and the fitted candidate analysis.

5.6 The quadratic genomic index

5.6.1 `run_qgsi()`

For genomic estimated breeding values with both linear and quadratic economic weights,

$$I = \mathbf{w}^\top \boldsymbol{\gamma} + \boldsymbol{\gamma}^\top \mathbf{W} \boldsymbol{\gamma},$$

with

$$\mathbb{E}(I) = \text{tr}(\mathbf{W}\boldsymbol{\Gamma}), \quad \text{Var}(I) = \mathbf{w}^\top \boldsymbol{\Gamma} \mathbf{w} + 2 \text{tr}(\mathbf{W}\boldsymbol{\Gamma}\mathbf{W}\boldsymbol{\Gamma}).$$

Accuracy and mean squared prediction error are reported only when `true_G` is supplied, which is possible in simulation. The package then requires $\mathbf{G}_{\text{true}} - \boldsymbol{\Gamma}$ to be positive semidefinite, since a prediction cannot vary more than what it predicts.

6 Evaluating and comparing indices

6.1 What a comprehensive comparison requires

A fair comparison uses one decision problem. Keep the candidates, trait order, trait direction, measurement scale, common genetic covariance and selected proportion fixed. Each family may need a different information covariance structure. Its source and estimation set must remain explicit. Use one definition of aggregate genetic merit when comparing R_{HI} or ΔH . Use the same validation split and simulation scenarios when those layers are added.

The analysis should answer four questions.

1. **Biological response:** what change is expected in every trait?
2. **Objective attainment:** how much of each target is reached, and which trait limits the result?
3. **Decision agreement:** do the methods rank and select the same candidates?
4. **Stability:** does the decision persist under covariance, weight and prediction uncertainty?

`comparison_objective()` defines one common desired-gain target, aggregate utility and genetic covariance. `compare_selection_methods()` then accepts results from `selection_index()`, `restricted_index()`, `generalized_index()`, `run_dgsi()`, and `run_qgsi()`. It reports fairness conditions, per-trait responses, attainment, Mahalanobis alignment, rank correlations and Jaccard similarities. Common validation values provide the same external yardstick for every selected set.

`compare_dg_and_qgsi()` remains a smaller descriptive helper. It compares candidate scores, ranks and selected sets from one desired-gain fit after iterative search and one quadratic genomic fit. Its `decision_summary` reports the selected counts, overlap and Jaccard similarity. The comparison is descriptive because the two methods represent different objectives. The *Multiple-trait selection* vignette gives explicit workflows for comparing the closed-form and iterative desired-gain fits, and for comparing linear and quadratic genomic indices.

```
comparison <- compare_selection_methods(  
  list(  
    Smith_Hazel = smith_hazel,  
    Pesek_Baker = pesek_baker,  
    Base = base_index,
```

```

    Rank_sum = rank_sum,
    Culling = culling
),
target_gains = desired_gains
)

comparison$fairness
comparison$responses
comparison$summary
comparison$rank_correlation
comparison$selected_jaccard

```

Read the per-trait response before any summary statistic. Then examine the worst target attainment. A mean can conceal failure of one critical trait. Finally, inspect rank and selected-set agreement. Similar average performance can still produce different operational decisions.

6.2 Match the evaluation to the method

Method	Primary evidence
Economic linear index	Per-trait response, R_{HI} , ΔH , relative efficiency, accuracy and weight sensitivity
Closed-form desired-gain index	Per-trait response, feasibility, target attainment and Mahalanobis alignment
Desired-gain index with iterative search	The same desired-gain criteria, selected differential, holdout objective and replicate stability
Restricted index	Per-trait response, restriction residual and loss under one common aggregate merit
Rank or threshold method	Passing rate where relevant, observed differential, rank stability and later-cohort validation
Quadratic genomic selection index	Linear and quadratic contributions, observed GEBV differential, approximate gain, validated nonlinear utility and simulation

One statistic cannot choose among all families. Each method must first satisfy the objective it was designed to represent.

6.3 The classical criteria

Let $I = \mathbf{b}^T \mathbf{x}$ denote the index score and $H = \mathbf{a}^T \mathbf{g}$ one fixed definition of aggregate genetic merit. The abbreviations below are not interchangeable measures of quality.

Criterion	Definition	Interpretation and limitation
R_{HI}	$\frac{\mathbf{b}^T \mathbf{G} \mathbf{a}}{\sqrt{\mathbf{b}^T \mathbf{P} \mathbf{b}} \sqrt{\mathbf{a}^T \mathbf{G} \mathbf{a}}}$	Correlation between index score (I) and aggregate genetic merit (H), bounded from -1 to 1. A negative value selects against the stated merit, zero indicates no linear association, and a value nearer 1 indicates closer ranking agreement. It is comparable only when every method uses the same $\mathbf{a}$.
ΔH	$i R_{HI} \sqrt{\mathbf{a}^T \mathbf{G} \mathbf{a}}$	Expected change in aggregate genetic merit, in the units defined by H . The software names it <code>delta_H</code> and abbreviates it as <code>dH</code> in compact print output. Positive values improve the stated merit. Compare magnitudes only under the same merit definition, population and intensity.
Δ_j	$i (\mathbf{G} \mathbf{b})_j / \sqrt{\mathbf{b}^T \mathbf{P} \mathbf{b}}$	Expected genetic response of trait j , in trait units or standard-deviation units according to the fitted space. Check every sign, magnitude and unit. This is the primary biological output.
Relative efficiency (RE)	$\Delta_m / (i \mathbf{G}_{mm} / \sqrt{\mathbf{P}_{mm}})$	Response in the declared main trait divided by its response under direct selection at the same intensity. $RE = 1$ matches direct response, $RE = 0.80$ retains 80%, $RE > 1$ exceeds it through correlated information, and $RE < 0$ moves the main trait in the unfavourable direction. It says nothing about attainment in the other traits.
h_I^2	$\mathbf{b}^T \mathbf{G} \mathbf{b} / \mathbf{b}^T \mathbf{P} \mathbf{b}$	Heritability of the index score treated as a composite trait, bounded from 0 to 1 when $\mathbf{P} - \mathbf{G}$ is valid. Its square root is the accuracy with which the score predicts its own additive genetic component. It does not measure agreement with the desired-gain objective.
Coefficient of variation of the index (CV_I)	$100 s_I / \bar{I} $	Standard deviation of the index scores as a percentage of their absolute mean. It changes when a constant is added to every score and is undefined when the index is centred. It is a descriptive legacy statistic, not a robust basis for ranking indices.

6.4 How each fails

Relative efficiency below 1 is not a defect. A multi-trait index deliberately gives up response in any single trait to gain elsewhere. Read it alongside what direct selection would have sacrificed.

R_{HI} requires a separately declared aggregate merit. Desired gains already define a valid response objective. They answer a different question. This distinction is a strength of desired-gain selection because breeders can state biological progress without inventing economic values. DesiredGainR still reports expected response, feasibility, attainment and stability. It returns NA only for R_{HI} and ΔH when aggregate merit remains undefined. If the programme supplies one common **aggregate_weights** vector, both criteria are reported under that common yardstick. The implied weights $\mathbf{w} = \mathbf{G}^{-1}\mathbf{P}\mathbf{G}^{-1}\mathbf{d}$ remain available as an explicit diagnostic through **implied_economic_weights()**. A separate implied vector for every direction would change the yardstick during comparison.

CV_I is undefined when the index is centred. Standardising or centring the traits puts the index mean at zero and the ratio diverges. The package withholds it and says so rather than reporting an enormous number.

h_I^2 does not require economic weights, but it measures prediction of the index's own genetic component. Two indices can have the same h_I^2 and point in different biological directions. Hence, it cannot replace assessment of trait responses and objective attainment.

6.5 A stronger decision panel for desired-gain indices

No classical scalar is sufficiently robust for a recommendation. Use the following panel.

1. Expected response Δ_j for every trait.
2. Exact-ray feasibility.
3. Worst-trait and joint-attainment probabilities for minimum floors.
4. Mahalanobis alignment, Satoh's β , and residual distance.
5. Rank correlation and selected-set overlap under perturbation.
6. Candidate selection probabilities after PEC propagation.
7. Cunningham information-deletion efficiency.
8. Diversity or coancestry over repeated cycles.

This panel separates biological attainment from statistical stability. A spectacular gain in one trait cannot conceal failure of another minimum, and a stable point estimate cannot conceal sensitivity to the estimated covariance matrices.

6.6 Reusing a fitted index

predict() applies a fitted index to a new candidate set, reusing the centring and scaling constants **from fitting** rather than recomputing them. That is deliberate: recomputing would score each cycle on its own mean, so a set that had advanced genetically would appear identical to one that had not.

The consequence is that new data must be on the same scale and in the same units. Where a trial has been re-standardised, refit rather than predict.

7 Uncertainty

7.1 Why this section is not optional

Every criterion above treats $\mathbf{G}$ and $\mathbf{P}$ as known. They are estimates, usually from fewer independent families than the covariance matrix has free parameters, and the coefficients are a non-linear function of them.

An index with coefficients that overlap zero differs from one whose coefficients are sharply determined. Yet the point estimate presents them identically.

7.2 index_uncertainty()

Resamples the covariance matrices and refits.

$$\mathbf{G}^* \sim \mathcal{W}_p(\nu_g, \mathbf{G}/\nu_g), \quad \mathbf{E}^* \sim \mathcal{W}_p(\nu_e, \mathbf{E}/\nu_e), \quad \mathbf{P}^* = \mathbf{G}^* + \mathbf{E}^*$$

The **genetic and residual** matrices are drawn, not the genetic and phenotypic ones, because $\mathbf{G}$ and $\mathbf{P}$ are built from the same records and drawing them independently would produce pairs that could not both be true.

7.2.1 Choosing the degrees of freedom

This is the input that governs every interval width, and the one users get wrong.

ν_g counts **independent genetic units**, not plots and not observations.

Design	ν_g is on the order of
Half-sib trial	Number of families
Diallel	Number of parents
Clonally replicated trial	Number of distinct clones

Replication within a genotype sharpens the residual matrix, not the genetic one. Supplying the plot count where the family count was meant understates every interval by roughly the square root of the replication.

7.2.2 Reading the result

For the desired-gain families the reported cost has an **exact reference point**. Since $\mathbf{G}\mathbf{b} = \mathbf{d}$ identically, an index fitted on the true $\mathbf{G}$ delivers the requested gains in the requested proportions. The cosine between $\mathbf{G}^*\hat{\mathbf{b}}$ and $\mathbf{d}$ therefore isolates estimation error with nothing confounding it.

An alignment interval whose lower bound sits well below one means the desired gains were specified more precisely than the data can deliver them. The honest response is to state fewer of them.

7.3 propagate_covariance_uncertainty()

Extends the same idea to a multi-cycle recommendation. Directions are re-evaluated across draws from the sampling distribution of the covariance, **rebuilding the founder population for each draw**, because $\mathbf{G}$ is the architecture the traits are built from rather than a matrix the simulation multiplies by. Each sampled residual matrix is passed directly to AlphaSimR’s phenotype model, so both genetic and residual uncertainty affect the cycles.

The result separates the two sources of uncertainty:

Source	Falls with	Remedy
Monte Carlo simulation	More replicates	Compute
Covariance estimation	Nothing the simulation can do	More independent genetic units in the trial

When the covariance component dominates, running more replicates is wasted effort, and the result says so.

8 Multi-environment structure

A multi-environment index is a single-environment index on an expanded trait set. `expand_environments()` builds the separable covariance

$$\text{Cov}(g_{je}, g_{kf}) = \mathbf{G}_{jk} \mathbf{C}_{ef}$$

where $\mathbf{C}$ is the between-environment genetic correlation matrix.

Mean off-diagonal of $\mathbf{C}$	Reading
Near 1	Environments rank genotypes almost identically. A single-environment index on trait means gives nearly the same answer at a fraction of the complexity.
Middling	Interaction is material and worth modelling
Near 0	Environments rank genotypes very differently. Model them as distinct target environments.

Environment-specific economic weights are supported: yield in a marginal environment serving a small area contributes less to net merit than yield in the main target environment.

Setting `include_stability = TRUE` returns per-trait response contrasts measuring departure from a genotype's own cross-environment mean. Pass `expanded$stability$constraint_matrix` to `restricted_index()`. A genotype whose deviations are uniformly zero performs at its own mean everywhere, which is what stability means here. Constraining it costs response in the mean. Use `restricted_index()` to price that trade rather than assume it.

The reported `interaction_share` is the heuristic one minus the mean environmental correlation. It is a screening summary, not a general interaction-variance identity.

9 Simulation evidence

Simulation tests a recommendation under explicit biological assumptions. It does not discover the breeding objective. The objective still comes from the breeder.

Chapter 13 of *Quantitative Genetics for Plant Breeding* motivates four design requirements. `DesiredGainR` implements them as follows.

1. `breeding_scenario()` records the programme, genetic architecture, evaluation method, and target environments.
2. `simulation_calibration()` compares target and realised variances, correlations, and heritabilities. It also checks the neutral marker panel.
3. `stress_test_desired_gains()` compares one desired-gain direction across plausible scenarios. It reports tail risks, minimum-attainment probability, Monte Carlo standard error, and regret.
4. Sequential batches stop when the requested Monte Carlo precision is reached or the maximum replication budget is used.

Test polygenic and oligogenic architectures. Use normal and gamma-distributed quantitative trait locus effects when both are plausible. Change one assumption at a time. Matched replicate seeds support paired comparisons. Exact alignment of random draws requires the same simulator call path.

Use `expand_environments()` to represent genotype by environment interaction through environment-specific genetic values and correlations. Different residual variances alone describe heteroscedasticity. They do not create genetic rank changes across environments.

A testcross scenario records general combining ability (GCA) as the source of selection information. `selection_information()` then links GCA records to the objective traits. HapBlockR remains responsible for parent choice, optimal contribution selection, cross choice, and mating allocation.

10 End-to-end pipeline walkthrough

10.1 Phase 1 — Evidence

Stage	Action	Function	Gate
1	Assemble candidate values, units and directions	—	Every trait has a stated direction and unit
2	Import or estimate G and P	<code>import_covariance()</code> , <code>estimate_genetic_covariances()</code>	Trait order verified, not summed
3	Check conditioning	<code>matrix_diagnostics()</code>	κ recorded. Standardise if large.
4	Check admissibility	automatic	P – G positive semidefinite
5	Repair if required	<code>bend_covariance()</code>	Adjustment recorded and reported

10.2 Phase 2 — Objective

Stage	Action	Function	Gate
6	State desired gains or weights	—	Desired gains are favourable magnitudes. Economic weights may have either sign.
7	Test attainability	<code>gain_feasibility()</code>	Required intensity within programme capability
8	Read the implied objective	<code>implied_economic_weights()</code>	No unexplained sign reversals
9	Test sensitivity	<code>weight_sensitivity()</code>	Selected set stable under plausible perturbation
10	Recover the historical objective	<code>retrospective_weights()</code>	Stated and revealed objectives compared

10.3 Phase 3 — Index and ranking

Stage	Action	Function	Gate
11	Fit the chosen family	<code>selection_index()</code> , <code>restricted_index()</code> , <code>run_dgsi()</code> , <code>run_qgsi()</code>	Method justified against the objective
12	Evaluate	<code>evaluate_index()</code> , <code>summary()</code>	Criteria within their natural bounds
13	Compare families	—	Overlap of selected sets reported, not only criteria
14	Verify constraints	<code>\$constraint</code>	Largest violation negligible

10.4 Phase 4 — Uncertainty and sign-off

Stage	Action	Function	Gate
15	Propagate covariance error	<code>index_uncertainty()</code>	ν_g counts genetic units, not plots
16	Look ahead, if relevant	<code>simulate_selection_cycles()</code> <code>optimize_desired_gains()</code>	Treated as research output
17	Record the decision	—	Provenance, repairs and overrides all documented

11 Interpreting diagnostics

11.1 Conditioning and stability

Symptom	Likely cause	Action
$\kappa(\mathbf{G}) > 10^4$ on raw scales	Trait scales differ by orders of magnitude	Standardise and refit
Coefficients change sign between families	Near-singular $\mathbf{G}$	Report conditioning. Prefer the base index.
Wide coefficient intervals	Few independent genetic units	State fewer desired gains
Alignment lower bound well below 1	Objective over-specified for the data	Reduce the number of constrained traits

11.2 Selection and shortfall

Symptom	Meaning
Fewer selected than requested under culling	Correct behaviour. The gates are hard.
Large empirical differential, small transmitted response	Selection acting on non-heritable variation
Two indices with different coefficients selecting the same lines	The choice between them does not matter here

12 Decision sign-off

Before a selection decision is approved, each of the following should have a recorded answer. An unanswered item means the analysis is incomplete rather than wrong.

1. What candidate set and data release were analysed?
2. What objective was declared, in what units, and before or after seeing the ranking?
3. Was feasibility checked, and was the required intensity attainable?
4. Do the implied economic weights carry the expected signs?
5. Was the covariance pair admissible, and was anything bent? By how much?
6. What is the condition number, and were traits standardised?
7. Which index family was used, and why that one?
8. Which benchmark methods were fitted under the same comparison conditions?
9. Which trait had the lowest target attainment?
10. Does the selected set survive a sensitivity analysis?
11. Was covariance uncertainty propagated, and with what ν_g ?

12. Is the reported response transmitted genetic response or an observed differential?
13. Were any overrides used, such as `allow_incompatible_estimated_P`?

13 Complete function map

13.1 Guide access

Function	Purpose
<code>open_desiredgain_guide()</code>	Open the installed HTML or PDF Breeder's Guide

13.2 Objective definition

Function	Purpose
<code>define_desired_gain_intervals()</code>	Record trait-specific acceptable gain intervals and their units
<code>implied_economic_weights()</code>	Weights implied by stated desired gains
<code>implied_desired_gains()</code>	Gains implied by stated weights
<code>gain_feasibility()</code>	Required intensity and selected fraction
<code>weight_sensitivity()</code>	Stability of the selected set under perturbation
<code>retrospective_weights()</code>	Objective implied by a past decision
<code>effective_weights()</code>	Weights an existing coefficient vector behaves as

13.3 Covariance handling

Function	Purpose
<code>estimate_genetic_covariance()</code>	Labelled approximations with recorded estimand
<code>import_covariance()</code>	Adapters for ASReml-R, sommer, breedR, BGLR, StageWise
<code>matrix_diagnostics()</code>	Eigenvalues, condition number, rank, definiteness
<code>bend_covariance()</code>	Repair to positive definiteness, with the adjustment reported

13.4 Index construction

Function	Purpose
<code>selection_index()</code>	Six classical index families and two operational comparators through one interface
<code>selection_information()</code>	Distinct information and objective vectors
<code>generalized_index()</code>	Economic or desired-gain index from P , C , and G
<code>restricted_index()</code>	Kempthorne–Nordskog, Tallis, Mallard, Harville, restricted Smith–Hazel
<code>restricted_breeding_values()</code>	Satoh projection and candidate β
<code>run_dgsi()</code>	Desired-gain index with iterative search and replicate diagnostics
<code>run_qgsi()</code>	Quadratic genomic selection index

Function	Purpose
<code>run_qgsi_desired_gain()</code>	Compatibility entry point that explains why desired gains require DGSI rather than QGSI
<code>run_dgsi_qgsi_pipeline()</code>	DGSI and QGSI fits with a descriptive candidate comparison
<code>compare_dg_and_qgsi()</code>	Scores, ranks and selected-set agreement for one DGSI and one QGSI fit

13.5 Evaluation and reuse

Function	Purpose
<code>evaluate_index()</code>	The standard criteria
<code>compare_selection_methods()</code>	Fairness checks, response attainment, ranking agreement and selected-set agreement
<code>comparison_objective()</code>	One target, utility and covariance for a cross-family comparison
<code>evaluate_restricted_response()</code>	Satoh progress, alignment, and residual distance
<code>index_information_efficiency()</code>	Cunningham deletion efficiency
<code>candidate_score_uncertainty()</code>	Score intervals and selection probabilities from PEC
<code>summary(), coef()</code>	Tidy tables of coefficients, response and criteria
<code>predict()</code>	Apply a fitted index to a new candidate set

13.6 Uncertainty

Function	Purpose
<code>index_uncertainty()</code>	Coefficient intervals, alignment, rank stability
<code>draw_covariance_pairs()</code>	Jointly admissible covariance draws
<code>propagate_covariance_uncertainty()</code>	Covariance error in a multi-cycle recommendation

13.7 Multi-environment

Function	Purpose
<code>expand_environments()</code>	Separable trait-by-environment covariance
<code>widen_environments()</code>	Reshape long-format trial data

13.8 Simulation

Function	Purpose
<code>breeding_scenario()</code>	Auditable biological and evaluation assumptions
<code>simulation_calibration()</code>	Target versus realised simulation checks
<code>stress_test_desired_gains()</code>	Scenario robustness, tail risk, and Monte Carlo precision
<code>founder_haplotypes()</code>	Phased marker data into AlphaSimR founders
<code>haplotypes_from_inbred_dosage()</code>	Dosage to haplotypes for inbred material
<code>dosage_diagnostics()</code>	Residual heterozygosity and missingness
<code>founder_population()</code>	Calibrate trait architecture to G

Function	Purpose
<code>simulate_selection_cycles()</code>	Forward simulation over cycles
<code>optimize_desired_gains()</code>	Search over desired-gain directions

13.9 Reference values

Function	Purpose
<code>rahimi_debnath_2023()</code>	Frozen published values for external validation

References

- Smith, H.F. (1936) A discriminant function for plant selection. *Annals of Eugenics* **7**, 240–250.
- Hazel, L.N. (1943) The genetic basis for constructing selection indexes. *Genetics* **28**, 476–490.
- Kempthorne, O. and Nordskog, A.W. (1959) Restricted selection indices. *Biometrics* **15**, 10–19.
- Tallis, G.M. (1962) A selection index for optimum genotype. *Biometrics* **18**, 120–122.
- Pesek, J. and Baker, R.J. (1969) Desired improvement in relation to selection indices. *Canadian Journal of Plant Science* **49**, 803–804.
- Mallard, J. (1972) La théorie et le calcul des index de sélection avec restrictions. *Biometrics* **28**, 713–735.
- Kemp, C.D. and Harville, D.A. (1975) Index selection with proportionality constraints. *Biometrics* **31**, 223–225.
- Yamada, Y., Yokouchi, K. and Nishida, A. (1975) Selection index when genetic gains of individual traits are of primary concern. *Japanese Journal of Genetics* **50**, 33–41.
- Elston, R.C. (1963) A weight-free index for ranking or selection with respect to several traits at a time. *Biometrics* **19**, 85–97.
- Cunningham, E.P. (1969) The relative efficiencies of selection indexes. *Acta Agriculturae Scandinavica* **19**, 45–48.
- Satoh, M. (2024) Characteristics of restricted selection indices and geometrical interpretation of restricted breeding values. *Journal of Animal Breeding and Genetics* **141**, 353–363.
- Beavis, W.D., Lamkey, K., Mahama, A.A. and Suza, W. (2023) Multiple Trait Selection. In *Quantitative Genetics for Plant Breeding*. Iowa State University Digital Press.
- Beavis, W.D., Mahama, A.A. and Suza, W. (2023) Multi Environment Trials: Linear Mixed Models. In *Quantitative Genetics for Plant Breeding*. Iowa State University Digital Press.
- Beavis, W.D., Mahama, A.A. and Suza, W. (2023) Simulation Modeling. In *Quantitative Genetics for Plant Breeding*. Iowa State University Digital Press.
- Higham, N.J. (2002) Computing the nearest correlation matrix. *IMA Journal of Numerical Analysis* **22**, 329–343.
- VanRaden, P.M. (2008) Efficient methods to compute genomic predictions. *Journal of Dairy Science* **91**, 4414–4423.
- Gaynor, R.C., Gorjanc, G. and Hickey, J.M. (2021) AlphaSimR: an R package for breeding program simulations. *G3* **11**, jkaa017.
- Rahimi, M. and Debnath, S. (2023) Estimating optimum and base selection indices in plant and animal breeding programs by development new and simple SAS and R codes. *Scientific Reports* **13**, 18977.